

Edge cases (a request that fails validation) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `sqlite_persistence_practice.py` (Edge cases section): "bad input never touches the database" is easy to believe and rarely checked. This section proves it with a real row count, then looks at the one case automatic checking can't cover — a valid request that fails between two writes. **If not:** the Build Task's "bad request body → 422, no database write" test case, and its started → completed/failed status updates, would rest on a guess.

Approaches 1-2 below assume your `sqlite_persistence` server is running on `localhost:8000` (or use `TestClient`, as noted). Read both depths — they're not "wrong, right," they're 2 real, valid ways to check the same thing, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way, with `curl`

```
$ sqlite3 runs.db "SELECT COUNT(*) FROM runs;"
0

$ curl -i -X POST localhost:8000/chat \
  -H "Content-Type: application/json" -d '{} '
HTTP/1.1 422 Unprocessable Entity
...
{"detail":[{"type":"missing","loc":["body","message"],
  "msg":"Field required", ...}]}
```

```
$ sqlite3 runs.db "SELECT COUNT(*) FROM runs;"
0
```

The row count before and after is identical — `0` both times (or whatever it already was). This confirms the requirement directly, with no extra code, using tools you already have.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a scripted check with `requests` and real assertions

Story: two `curl` outputs checked by eye prove it once, today. A script with `assert` proves it every time you run it, and stops loudly the day it breaks. **If not:** a future change that accidentally writes before checking input would go unnoticed.

```
# practice/sqlite_persistence_practice.py – Edge cases section
import sqlite3
import requests

conn = sqlite3.connect("runs.db")

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()

url = "http://localhost:8000/chat"
response = requests.post(url, json={})
# why: assert stops the script with this message if the check fails
assert response.status_code == 422, f"expected 422, got {response.status_code}"

after = row_count()
assert before == after, f"row count changed: {before} -> {after}"

print("422 confirmed, no row written")
print(response.json())
```

Expected output (the second line is shown wrapped onto 2 lines just to fit the page — really one line of output):

```
422 confirmed, no row written
{'detail': [{'type': 'missing', 'loc': ['body', 'message'],
  'msg': 'Field required', 'input': {}}]}
```

Approach 2 — the same check with `TestClient`, no server running

Story: Approach 1 needs uvicorn running in another terminal. `TestClient` (from `health_route`) runs the same check inside one `python` command. **If not:** the Build Task's `test_api.py` would be the first time you checked a 422 without a live server.

```
# practice/sqlite_persistence_practice.py – Edge cases section
# (add this at the bottom of the Real-world code, which already
# defines app and conn)
from fastapi.testclient import TestClient

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

if __name__ == "__main__":
    client = TestClient(app)
    before = row_count()
    response = client.post("/chat", json={})
    assert response.status_code == 422
    assert row_count() == before
    print("422 confirmed, no row written")
```

Expected output (`python sqlite_persistence_practice.py`):

```
422 confirmed, no row written
```

Approach 3 — two writes per request, kept all-or-nothing with a transaction

Story: a route that writes a "started" row, runs the logic, then updates the row to "completed" has a gap in the middle. If the logic crashes there, the "started" row is left behind — and the request was perfectly valid, so no 422 ever fires. `with conn:` makes both writes one unit. **If not:** the Build Task's `runs` table would be the first place you ever used `lastrowid` and `UPDATE`, and the first time you had to think about what a crash between two writes leaves behind.

```
# practice/sqlite_persistence_practice.py – Edge cases section
import sqlite3
from fastapi import FastAPI, HTTPException
from fastapi.testclient import TestClient
from pydantic import BaseModel

app = FastAPI()
# why: a separate file, so this table's extra "status" column
# never clashes with the Real-world runs.db table
conn = sqlite3.connect("two_step.db", check_same_thread=False)
conn.execute("""
    CREATE TABLE IF NOT EXISTS runs (
```

```

        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        status TEXT
    )
    """

class ChatRequest(BaseModel):
    message: str

def run_chat_two_step(message: str) -> str:
    # why: a fake crash between the two writes, on command
    if message == "fail-after-insert":
        raise RuntimeError("simulated failure between steps")
    return "You said: " + message

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

@app.post("/chat")
def chat(request: ChatRequest):
    try:
        # how: a transaction – commits if the block finishes,
        # rolls back BOTH writes if anything inside raises
        with conn:
            cursor = conn.execute(
                "INSERT INTO runs (input, status) VALUES (?, ?)",
                (request.message, "started"),
            )
            # how: the id SQLite just gave the new row
            run_id = cursor.lastrowid
            reply = run_chat_two_step(request.message)
            conn.execute(
                "UPDATE runs SET output = ?, status = ? WHERE id = ?",
                (reply, "completed", run_id),
            )
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    return {"reply": reply}

if __name__ == "__main__":
    client = TestClient(app)
    before = row_count()
    response = client.post("/chat", json={"message": "fail-after-insert"})
    print(response.status_code)          # expected: 500
    print(row_count() == before)         # expected: True

```

```
response = client.post("/chat", json={"message": "hi"})
print(response.status_code)      # expected: 200
print(row_count() == before + 1) # expected: True
```

Expected output (`python sqlite_persistence_practice.py`):

```
500
True
200
True
```

Without `with conn:`, the `INSERT` (status `"started"`) could be committed on its own before `run_chat_two_step()` raised, leaving a row stuck at `"started"` forever — a silent partial write, from a completely valid request.

Difference from Basic: Approach 1 turns a one-time manual check into a real, repeatable assertion. Approach 2 runs the same check in-process with `TestClient`, so no server is needed. Approach 3 tackles a different failure — not bad input at all, but a *valid* request that fails between two writes — and shows the real fix: a transaction (`with conn:`), which no amount of request checking could provide.

Which one should you actually write? Approach 1 or 2 is the right amount of testing for a single-write route — confirm the 422 case with a real assertion and move on. Reach for Approach 3 the moment one request makes more than one database write. The Build Task's `runs` table uses the same `INSERT` → `lastrowid` → `UPDATE` pieces, but makes one deliberate change: it *commits* the `"started"` row first (its own `with conn:`), then updates it to `"completed"` or `"failed"` in a second one — because there, a crashed run should leave an honest `"failed"` record you can look up, not vanish.